

Real-world (a tool backed by a real API call) — Solution

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Basic Version

Approach 1 — the direct way

```
# real_api_tool_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI
from http_client import request_with_retry # your Doc02 client

WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}

@tool
def get_weather(city: str) -> str:
    """Get the current temperature for a named city."""
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return f"No coordinates on file for {city}."

    params = {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }
    response = request_with_retry(WEATHER_URL, params=params)
    temperature = response["current_weather"]["temperature"]
    return f"Currently {temperature}°C in {city}"

model_with_tools = ChatOpenAI().bind_tools([get_weather])
```

```
response = model_with_tools.invoke("What's the weather in Lahore?")
print("Requested tool calls:", response.tool_calls)

for call in response.tool_calls:
    if call["name"] == "get_weather":
        result = get_weather.invoke(call["args"])
        print("Real result:", result)
```

Expected output:

```
Requested tool calls: [{'name': 'get_weather', 'args': {'city': 'Lahore'}, 'id':
Real result: Currently 34.2°C in Lahore
```

This version makes a real network call through your existing retry logic, and returns a plain text summary. It only knows 2 cities, and it doesn't catch a shape-mismatch or exhausted-retries failure — both worth fixing next.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

Approach 1 — params-building split out, and checked

```
# real_api_tool_practice.py
from langchain_core.tools import tool
from langchain_openai import ChatOpenAI

from http_client import request_with_retry

WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

CITY_COORDINATES = {
    "lahore": {"latitude": 31.55, "longitude": 74.34},
    "karachi": {"latitude": 24.86, "longitude": 67.01},
}

def build_weather_params(city: str) -> dict | None:
    coords = CITY_COORDINATES.get(city.lower())
    if coords is None:
        return None
    return {
        "latitude": coords["latitude"],
        "longitude": coords["longitude"],
        "current_weather": True,
    }

@tool
def get_weather(city: str) -> str:
    """Get the current temperature for a named city."""
    params = build_weather_params(city)
    if params is None:
        return f"No coordinates on file for {city}."

    response = request_with_retry(WEATHER_URL, params=params)
    temperature = response["current_weather"]["temperature"]
    return f"Currently {temperature}°C in {city}"
```

```
def get_model_with_tools() -> ChatOpenAI:
    return ChatOpenAI(model="gpt-4o-mini", temperature=0).bind_tools([get_weather])

def main() -> None:
    model_with_tools = get_model_with_tools()
    response = model_with_tools.invoke("What's the weather in Lahore?")

    if not response.tool_calls:
        print("The model answered directly, without calling a tool.")
        print(response.content)
        return

    for call in response.tool_calls:
        if call["name"] == "get_weather":
            result = get_weather.invoke(call["args"])
            print(f"Called get_weather({call['args']}) -> {result}")
        else:
            print(f"Unexpected tool requested: {call['name']}")

if __name__ == "__main__":
    main()
```

Expected output:

```
Called get_weather({'city': 'Lahore'}) -> Currently 34.2°C in Lahore
```

Difference from Basic: pulling `build_weather_params()` out as its own function makes the "do we have coordinates for this city" check independently testable, without needing a real network call or a running model. Checking `response.tool_calls` before looping over it also matters here for the same reason it mattered in the Basic exercise: an obvious-looking question is still not a *guarantee* the model calls anything. Neither version yet defends against the API's response not having the shape it expects, or supports a city outside the fixed dictionary — that's what Advanced adds.

[Back to the exercise](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Approach 1 — safe nested access instead of a `KeyError` crash

```
# real_api_tool_practice.py
def get_weather(city: str) -> str:
    params = build_weather_params(city)
    if params is None:
        return f"No coordinates on file for {city}."

    response = request_with_retry(WEATHER_URL, params=params)
    temperature = response.get("current_weather", {}).get("temperature")

    if temperature is None:
        return f"Got a response for {city}, but it didn't include a temperature."

    return f"Currently {temperature}°C in {city}"
```

Expected output on a normal response:

Currently 34.2°C in Lahore

Expected output if the API's response shape ever changes and `current_weather` is missing (simulated by passing a response dict like `{}`):

Got a response for Lahore, but it didn't include a temperature.

This is the difference between a shape mismatch crashing your whole tool with a `KeyError` traceback, and it becoming a clear, returned string the model (and next exercise's failure-handling pattern) can actually react to — the exact same philosophy as Core Concepts' "getting a tool's failure back to the model, correctly," applied here to a malformed *success* response instead of a raised exception.

Approach 2 — a real geocoding call instead of a 2-city lookup table

```
# real_api_tool_practice.py
GEOCODING_URL = "https://geocoding-api.open-meteo.com/v1/search"
```

```

WEATHER_URL = "https://api.open-meteo.com/v1/forecast"

def geocode_city(city: str) -> dict | None:
    response = request_with_retry(GEOCODING_URL, params={"name": city, "count": 1})
    results = response.get("results")
    if not results:
        return None
    first_match = results[0]
    return {"latitude": first_match["latitude"], "longitude": first_match["longitude"]}

@tool
def get_weather(city: str) -> str:
    """Get the current temperature for any named city, anywhere."""
    coords = geocode_city(city)
    if coords is None:
        return f"Couldn't find a location named {city}."

    params = {**coords, "current_weather": True}
    response = request_with_retry(WEATHER_URL, params=params)
    temperature = response.get("current_weather", {}).get("temperature")

    if temperature is None:
        return f"Got a response for {city}, but it didn't include a temperature."

    return f"Currently {temperature}°C in {city}"

```

Expected output for a city that was never hardcoded anywhere:

```
Currently 15.9°C in Tokyo
```

Now the tool's implementation actually matches what its description promises — "for any named city" — instead of silently only working for the 2 entries someone happened to type into a dictionary. This costs a second real network call (through the same `request_with_retry()` client) for every request, which is a real, worthwhile tradeoff for correctness here.

Difference from Intermediate, and between these 2 Advanced approaches: Intermediate's tool works correctly for exactly the 2 cities it knows about, and crashes on any response shape it doesn't expect. Approach 1 fixes the crash — a malformed response becomes a clear message instead of an unhandled exception. Approach 2 fixes the coverage gap — replacing the fixed lookup table with a real geocoding call so the tool's description and its actual behavior finally match for any city. The two are independent fixes for two different problems, and a real version of this tool would want both at once.

Which one should you actually write? Approach 1's safe-access pattern (`.get(...).get(...)`, with a clear fallback message) is worth adding to essentially any tool that parses a real API's JSON response — it's cheap and it's exactly the failure-shape the next exercise teaches you to design for on purpose. Approach 2's geocoding call is worth adding once the tool's own description makes a promise ("any city") that a hardcoded lookup table can't actually keep — for a demo where 2-3 known cities are genuinely all you need, the fixed dictionary from Intermediate is simpler and fine.